

Instructor: Date:

Learner: Regional Branch:

Program: Center:

CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

Prevention is usually the best option to avoid errors.



Learning Outcome:

Understand key information about the concepts, principles, and methodologies in simple spoken and written situations. Talk about programming techniques, use cases, kinds, and sequences in simple spoken and written situations.

Ask your partner:

- What are the most common problems a developer may find in software programming?
- What information is necessary for a developer in order to understand and detect a software problem?

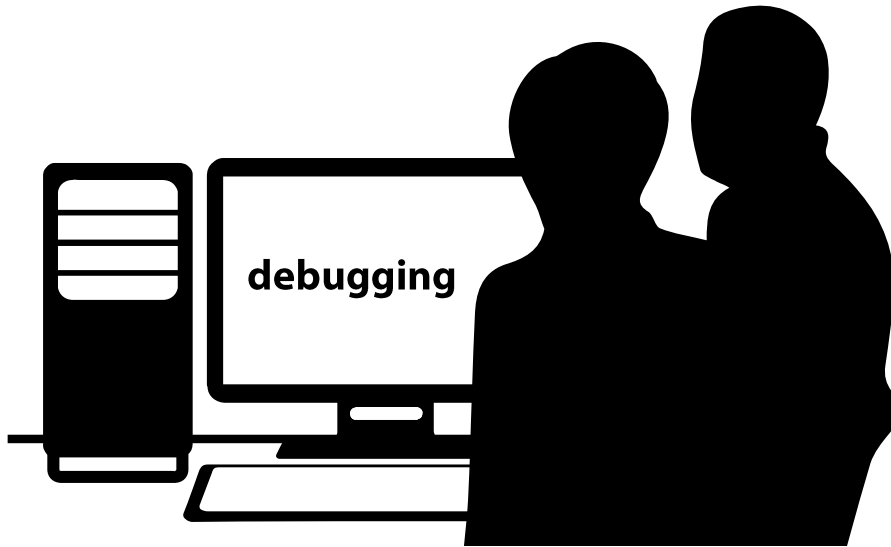
Can Do Statements:

- I can obtain information about bugs and debugging.
- I can explain different types of errors related to bugs and debugging.
- I can identify key features about bugs and debugging.



Let's Explore!

Scott helps Grace to understand some facts about bugs and debugging.



Instructions: Read the text below.

- Grace:** I'm having difficulties with this project. I'm trying to finish with this programming language but I couldn't. I decided not to continue until you can help me.
- Scott:** Thanks. What I see here is that while you are programming, you can find that actual results vary from expected results and there is an unwanted behavior. This is a bug also called defect or incident. Most bugs are almost always due to human errors in source code or its design.
- Grace:** I see what you mean. I'm making mistakes during the process, right?
- Scott:** Yes, a bug is a coding error in a computer program. There are two main types of errors, syntax and logic errors.
- Grace:** I know what you mean, I am learning to identify them. For example, a syntax error is in the source code of a program. This error is in language's rules for constructing statements or from using the wrong symbol like parentheses.
- Scott:** Right. since computer programs have to follow strict syntax to compile correctly, any aspect of the code that doesn't match the syntax of the programming language will constantly produce a syntax error.
- Grace:** Can you explain me how to identify logic errors?
- Scott:** Well, the logic errors are defects that occur in a program's source code that results in incorrect or unexpected behavior. To spot these errors you need to check the flow or logic sequence of the program.
- Grace:** At college, my teacher asks me to learn about debugging.
- Scott:** Debugging is the process of finding and fixing bugs. The process usually starts after writing the code and continues in successive stages as code is combined with other units of programming.
- Grace:** I guess defect prevention is very effective and efficient in reducing the number of bugs, that's why it's important to conduct defect discovery and removal.





Let's Practice!

Sometimes new topics are a little hard to learn.

Instructions: Complete the sentences by writing its corresponding word. Use capital letter where necessary. Use: almost always, constantly and usually

1. Most bugs are _____ due to typing human errors in the source code or in its design.
2. Bugs _____ appear when results vary from expected outcomes.
3. Behavior in programming must be _____ the same, except when there is an unexpected conduct.



Let's Practice!

Let's check some important definitions

Instructions: Find the word in the wordsearch below. Read each clue and circle the word.

1. Defect _____ is a very effective and efficient of reducing the number of bugs.
2. _____ fixes problems so that the program works correctly.
3. A _____ is a defect or incident.
4. An error in the source code of a program is called _____.
5. Defects that are occurring in a program's source code that results in incorrect or unexpected behavior are called a _____ error.

M	V	Q	O	Z	I	B	L	T	A	U	U	H	G	H	R
O	P	R	E	V	E	N	T	I	O	N	P	X	E	S	I
T	U	N	E	A	R	N	E	D	D	T	B	A	G	C	G
E	C	I	T	A	S	P	O	C	B	S	I	O	E	L	I
R	D	Y	I	T	R	E	N	D	Y	K	D	T	E	C	N
T	D	I	A	B	U	G	I	T	Y	L	G	T	V	K	A
H	E	N	I	P	L	E	H	S	O	V	L	C	I	X	L
H	B	N	I	N	T	E	R	E	S	T	L	C	I	X	L
E	U	B	Q	Y	T	K	L	I	R	P	R	E	D	N	O
M	G	Q	O	Z	I	B	L	T	A	U	U	H	G	H	R
O	G	E	P	L	I	C	A	T	E	E	P	X	E	S	I
T	I	N	E	A	R	N	E	D	D	T	B	A	G	C	G
E	N	I	T	A	S	S	Y	N	T	A	X	O	E	L	I
R	G	Y	I	T	R	E	N	D	Y	K	D	T	E	C	N
T	L	I	A	B	I	L	I	T	Y	L	G	T	V	K	A
H	G	N	I	P	L	E	H	S	O	V	L	C	I	X	L



Extension Activity

I look for the best strategy to avoid bugs.

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Search for information about bugs and how to avoid them.
- Identify different useful strategies in order to avoid them.
- Design a brochure presenting that information.
- Share the brochure with the class and discuss about it.

Self-Assessment

- | | Yes | No |
|--|-----------------------|-----------------------|
| 1. I deepen my knowledge about bugs. | ☐ | ☐ |
| 2. I discover strategies to avoid bugs. | ☐ | ☐ |
| 3. I present information with a brochure. | ☐ | ☐ |
| 4. I exchange information with classmates. | ☐ | ☐ |



Target Words

- **compile** (verb)
to put information together in a document or text
e.g. Since computer programs have to follow strict syntax to compile correctly, any aspect of the code that doesn't match the syntax of the programming language will constantly produce a syntax error.
- **unexpected** (adjective)
a result that was not the one someone expects
e.g. The logic errors are defects that occur in a program's source code that results in incorrect or unexpected behavior.
- **conduct** (verb)
to carry out an activity
e.g. I guess defect prevention is very effective and efficient in reducing the number of bugs, that's why it's important to conduct defect discovery and removal.
- **defect** (noun)
an imperfection or problem in software programming
e.g. What I see here is that while you are programming, you can find that actual results vary from expected results and there is an unwanted behavior. This is a bug also called defect or incident.
- **spot** (verb)
in this case, to detect or identify an error in the system
e.g. To spot these errors you need to check the flow or logic sequence of the program.
- **symbol** (noun)
the different figures and marks used in software programming
e.g. This error is in a language's rules for constructing statements or from using the wrong symbol like parentheses.



Learning Points

Adverbs of Frequency, Usually, Constantly, Almost Always

In English, we use the **adverbs of frequency** to mention how frequently we do an activity or action.

The **adverbs** can modify the meaning of a sentence by saying how often it happens.

Generally the **adverbs of frequency** use a percentage to show the frequency, see the chart below.

An **adverb of frequency** goes before the main verb of the sentence except the verb to be

For example:

*A developer **constantly** finds bugs.*

*I **almost always** make mistakes when programming.*

When you use an auxiliary verb the **adverb** goes in the middle of the auxiliary and the main verb

For example:

*Bugs and defects in the system are **usually** prevented by running debugging.*

FREQUENCY	ADVERB
70%	Constantly
80%	Usually
90%	Almost Always

Examples

- *Most bugs are almost always due to human errors in source code or its design.*
- *Since computer programs have to follow strict syntax to compile correctly, any aspect of the code that doesn't match the syntax of the programming language will constantly produce a syntax error.*
- *Debugging is the process of finding and fixing bugs. The process usually starts after writing the code and continues in successive stages as code is combined with other units of programming.*





Learning Points

Infinitives After Certain Verbs, Ask - Learn - Explain – Decide in Affirmative, Negative and Questions

In English, if you need to write a verb with another action or activity, you usually use an **infinitive**. following the main verb. It allows you to express purpose in a sentence.

After the following verbs :ask, learn, explain and decide, we always have to use the infinitive form of another verb (to+infinitive), to introduce another action or activity.

Also, after the following verbs: advice, ask, encourage, invite, order, persuade, remind, tell, warn, expect, intend, would prefer, want, would like, we can use the infinitive form of another verb.

For example:

*I am learning **to** detect bugs easier.*

*She decided **to** eliminate all the defects.*

If you want to create a sentence in negative, follow the next structure:

Subject + main verb + (object) + verb in infinitive + complement

For example:

*You asked her **not** to erase any bugs.*

*We decided **not** to update the system.*

In questions, use this:

(Wh word) + Aux. verb (Do-does-did) + subject + main verb + (object) + verb in infinitive + complement

For example:

Does she learn to program applications?

Why do you ask him to leave the platform?

Look at the following chart:

Verb	Infinitive
Ask	Ask to go, ask to leave
Learn	Learn to create, learn to play
Explain	Explain to write, explain to update
Decide	Decide to check, decide to get

Examples

- I'm having difficulties in this project. I try to finish with this programming language but I cannot. I decide not to continue until you can help me.*
- Can you explain me to identify logic errors?*
- Sure. I am learning to identify them. For example, a syntax error is in the source code of a program.*